package.json & Semantic Versioning

Why it matters

`package.json` is the manifest at the heart of every Node project — it names the project, lists its dependencies, defines its scripts, and controls how the package is published and consumed. Get it right and installs are reproducible, upgrades are predictable, and CI is green. Get it wrong and you ship the classic disasters: "works on my machine," a dependency that silently jumped a major version and broke production, or a deploy that pulled different code than your laptop did. Almost every reproducibility and supply-chain incident traces back to a misunderstanding of two things in this file: version ranges (`^` vs `~`) and the role of the lockfile.

Semantic Versioning (semver) is the contract that makes ranges meaningful. It is a shared promise between package authors and consumers about what a version number change implies. Interviewers love this topic because it is where "I can install packages" becomes "I understand why my build is or isn't reproducible."

The concept, simply

A semver version has three numbers: **MAJOR.MINOR.PATCH**, e.g. `4.18.2`. The promise is:

- **MAJOR** — incremented for a *breaking* change. Upgrading may require code changes on your side.
- **MINOR** — new functionality added in a *backward-compatible* way. Safe to take.
- **PATCH** — a backward-compatible *bug fix*. Safe to take.

In `package.json` you rarely pin one exact version. Instead you write a *range* — a rule for which versions are acceptable — using prefixes like `^` (caret) and `~` (tilde). The range

says "install the newest version that satisfies this rule." A separate *lockfile* then records the exact version that was actually installed, so everyone and every deploy gets byte-identical dependencies.



Semver encodes intent — major means breaking, minor means new-and-safe, patch means fix — and range operators like caret and tilde decide how far npm may upgrade automatically.

How it works

Key `package.json` fields.

- `name` and `version` — identity; required to publish. `version` must be valid semver.
- `type` — `"module"` or `"commonjs"`; decides how `.js` files are interpreted.
- `main` / `exports` — the package's entry point(s) (see module-resolution).
- `scripts` — named commands run with `npm run` (see npm-and-scripts).
- `dependencies` — packages your code needs at *runtime*; installed for anyone who installs your package.
- `devDependencies` — packages needed only to *develop/build/test* (bundlers, test runners, TypeScript, linters). Skipped when your package is installed as someone else's dependency, and skippable in production with `npm ci --omit=dev`.
- `peerDependencies` — packages the host app must provide (e.g. a React plugin declares `react` as a peer so it shares the app's single copy).

- `optionalDependencies` — install failures are tolerated (platform-specific binaries).
- `engines` — declares supported Node/npm versions; a guardrail against running on an unsupported runtime.
- `files` — allowlist of what actually gets published to npm; keeps the tarball lean.
- `bin` — maps a command name to an executable file, powering CLIs and `npx`.

Range operators against a base version (say `1.2.3`).

- `^1.2.3` (**caret**) — allows changes that do not modify the left-most non-zero digit: `≥1.2.3 <2.0.0`. Takes new minors and patches, not a new major. This is npm's default when you `npm install pkg`. Special zero-rules: `^0.2.3` means `≥0.2.3 <0.3.0` (minor is treated as breaking when major is 0), and `^0.0.3` means exactly `0.0.3` patch range `≥0.0.3 <0.0.4`.
- `~1.2.3` (**tilde**) — allows patch-level changes if a minor is specified: `≥1.2.3 <1.3.0`. Takes patches only.
- `1.2.3` (**exact**) — only that version.
- `1.2.x` / `1.2` — any patch of `1.2`. `1.x` — any minor/patch of major 1.
- `≥1.2.3 <2.0.0` — explicit compound range (what caret expands to).
- `*` or `latest` — any/newest version; unreproducible, avoid.

The lockfile is the source of truth for installs. Ranges describe what is *allowed*; the lockfile records what was *chosen*. npm writes `package-lock.json`, yarn writes `yarn.lock`, pnpm writes `pnpm-lock.yaml`. It captures the exact resolved version and integrity hash of every package in the entire tree — including transitive dependencies. Commit it. Two commands behave very differently:

- `npm install` — may update the lockfile to newer versions still satisfying your ranges, and can mutate `package.json`.
- `npm ci` — installs *exactly* what the lockfile says, fails if `package.json` and the lockfile disagree, and deletes `node_modules` first. This is the command for CI and production because it is deterministic.

The caret's real rule: the left-most non-zero digit. The single sentence that makes caret behaviour predictable is "allow anything that does not change the left-most non-zero part of the version." For a normal `1.x.y` release the left-most non-zero digit is the

operator quietly tightens. This is not npm being inconsistent — it is one rule applied uniformly, and pre-1.0 packages lean on it because SemVer explicitly says `0.y.z` offers no stability guarantee, so every `0.x` bump is treated as potentially breaking. The table makes the shift concrete:

Range	Expands to	Takes
`^1.2.3`	`≥1.2.3 <2.0.0`	minor + patch
`^0.2.3`	`≥0.2.3 <0.3.0`	patch only (minor is treated as breaking)
`^0.0.3`	`≥0.0.3 <0.0.4`	nothing — effectively pinned
`~1.2.3`	`≥1.2.3 <1.3.0`	patch only
`~1.2`	`≥1.2.0 <1.3.0`	patch only
`~1`	`≥1.0.0 <2.0.0`	minor + patch

Pre-release tags sit outside the ordinary ranges. A version can carry a pre-release suffix such as `2.0.0-beta.1` or `2.0.0-rc.2`, and SemVer orders these *below* the matching stable release: `2.0.0-beta.1 < 2.0.0-rc.2 < 2.0.0`. Crucially, npm will not hand a pre-release to a plain range by default — `^1.2.3` never selects `2.0.0-beta.1`, and even `^2.0.0-beta.1` only matches other `2.0.0` pre-releases, not later ones — precisely so an unfinished build cannot sneak into a normal install. You opt in explicitly, e.g. `npm install pkg@next` or a range like `2.0.0-rc.2`. Build metadata after a `+` (`1.4.0+20240115`) is ignored entirely for ordering. This is why a teammate on a beta and a teammate on the range can end up on genuinely different code even though both "satisfy" the intent — another reason the committed lockfile, not the range, is the real record of what shipped.

Why ranges + lockfile together. Ranges let you accept safe upgrades when you deliberately run `npm update`; the lockfile guarantees that, until you do, everyone installs identical code. Without a lockfile, `^4.18.2` could resolve to `4.18.2` today and

the range would allow drift.

Code examples

1. A realistic `package.json`.

JSON Copy

```
{
  "name": "my-api",
  "version": "1.4.0",
  "type": "module",
  "engines": { "node": "≥20" },
  "main": "./dist/index.js",
  "scripts": {
    "dev": "node --watch src/index.js",
    "build": "tsc -p tsconfig.json",
    "test": "node --test"
  },
  "dependencies": {
    "express": "^4.18.2",
    "zod": "~3.23.8"
  },
  "devDependencies": {
    "typescript": "^5.5.0"
  },
  "files": ["dist"]
}
```

2. What each range accepts.

JAVASCRIPT Copy

```
// Given published versions: 1.2.3, 1.2.9, 1.3.0, 1.9.5, 2.0.0
// "^1.2.3" → highest of 1.2.3 .. 1.9.5 (any minor/patch, not 2.x) → 1.9
// "~1.2.3" → highest of 1.2.3 .. 1.2.9 (patch only, not 1.3.0)    → 1.2
```

3. Reproducible install in CI.

JAVASCRIPT Copy

```
// package.json + package-lock.json are committed together.
// CI / Dockerfile:
//   npm ci                 // exact lockfile install, deterministic
//   npm ci --omit=dev      // production: skip devDependencies
// NOT `npm install` in CI — it may silently bump versions.
```

4. Pinning when you cannot risk any drift.

JAVASCRIPT Copy

```
// Save an exact version instead of a caret range:
//   npm install lodash --save-exact   // writes "lodash": "4.17.21"
// Or set once in .npmrc:  save-exact=true
```

Common mistakes & gotchas

- **Not committing the lockfile.** Installs stop being reproducible; teammates and CI can get different transitive versions.
- **Using `npm install` in CI.** It may update versions and the lockfile. Use `npm ci`.
- **Confusing dependencies with devDependencies.** Putting a build tool in `dependencies` bloats production installs; putting a runtime library in `devDependencies` makes production crash with "module not found."
- **Assuming `^0.x` behaves like `^1.x`.** With a `0` major, caret does NOT allow minor bumps — `^0.2.0` stays within `0.2.x` because pre-1.0 packages treat every minor as potentially breaking.
- **Trusting semver blindly.** Authors make mistakes; a "patch" can break you. That is exactly why the lockfile plus a test suite matter.

- **Editing versions by hand and forgetting the lockfile.** The two drift apart; `npm ci` then refuses to install.
- **Ignoring `peerDependencies` warnings.** Mismatched peers cause duplicate copies and subtle runtime bugs.

Best practices

- Always commit the lockfile and treat it as reviewed source.
- Use `npm ci` (not `npm install`) in CI and Docker builds; add `--omit=dev` for production images.
- Classify dependencies correctly: runtime → `dependencies`, tooling → `devDependencies`, host-provided → `peerDependencies`.
- Keep the caret default for most deps (safe with a lockfile), but pin exact versions for anything security- or behaviour-critical, or use tilde for cautious patch-only updates.
- Set `engines.node` so an unsupported runtime fails fast.
- Add a `files` allowlist (or `.npmignore`) so you publish only what is needed.
- Upgrade deliberately with `npm update` / `npm outdated`, then run tests, then commit the new lockfile.
- Never hand-edit `package-lock.json`; let the CLI manage it.

⭐ Interview lens

Q: Explain `^1.2.3` vs `~1.2.3`. Caret allows minor and patch updates: `≥1.2.3 <2.0.0`. Tilde allows patch updates only (when a minor is given): `≥1.2.3 <1.3.0`. Caret is npm's default and takes more updates; tilde is more conservative.

Q: If `package.json` pins ranges, why do we also need a lockfile? Ranges say what is *allowed*, not what is *installed*. Without a lockfile, `^4.18.2` can resolve to different actual versions over time and across machines. The lockfile records the

exact resolved version and integrity hash of every package (including transitive deps), making installs byte-reproducible.

Q: Difference between `npm install` and `npm ci`? `npm install` resolves ranges, may update the lockfile and `package.json`, and installs on top of existing `node_modules`. `npm ci` wipes `node_modules` and installs the lockfile exactly, failing if it disagrees with `package.json`. Use `ci` for deterministic CI/production installs.

Q: dependencies vs devDependencies vs peerDependencies? `dependencies` ship to consumers and are needed at runtime. `devDependencies` are for building/testing and are not installed when your package is a dependency of another. `peerDependencies` declare something the host app must supply, so a plugin shares the app's single copy (e.g. React) instead of bundling its own.

Q: Trap — what does `^0.2.3` allow? Only `≥0.2.3 <0.3.0`. When MAJOR is 0, caret does not permit minor bumps because pre-1.0 packages may break in any minor. Many candidates wrongly say it allows up to `<1.0.0`.

Q: A "patch" upgrade broke production. What went wrong and how do you prevent it? Semver is a promise authors can violate; a patch release contained a breaking change. Prevention: a committed lockfile + `npm ci` so upgrades only happen when you choose, plus a test suite run before adopting the new lockfile. Consider pinning exact versions for critical packages.

Cheat-card

- **Semver:** MAJOR = breaking · MINOR = new+safe · PATCH = fix+safe.
- **^1.2.3** → `≥1.2.3 <2.0.0` (minor+patch, npm default).
- **~1.2.3** → `≥1.2.3 <1.3.0` (patch only).

- **deps** = runtime · **devDeps** = build/test · **peerDeps** = host-provided.
- **Lockfile** pins exact tree + hashes — commit it.
- `npm ci` = deterministic lockfile install for CI/prod (`--omit=dev`).

QUICK CHECK — no marks, just for you.

1. For a base version, what does the caret range `^1.2.3` allow?

- ◯ Only exactly 1.2.3
- ◯ Patch updates only (>=1.2.3 <1.3.0)
- ◯ Minor and patch updates (>=1.2.3 <2.0.0), but not a new major
- ◯ Any version including 2.x

Check answer

2. Given ranges already pin versions, why is a committed lockfile still needed?

- ◯ Ranges describe what is installed exactly
- ◯ The lockfile replaces package.json entirely
- ◯ Lockfiles are only for publishing packages
- ◯ Ranges say what is allowed, not what is installed; the lockfile records exact resolved versions and hashes for reproducible installs

Check answer

3. What does `^0.2.3` allow, and why does it differ from `^1.2.3`?

Only >=0.2.3 <0.3.0 (patch only), because with major 0 the left-most non-zero digit is the minor, treated as breaking

- ○ >=0.2.3 <1.0.0, the same rule as a 1.x caret
- ○ Any 0.x version
- ○ Exactly 0.2.3 with no updates

Check answer

‹ Previous
Module Resolution

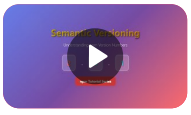
Next ›
npm & Scripts

LEARN MORE

ARTICLES & DOCS

- package.json
  docs.npmjs.com
- About semantic versioning
  docs.npmjs.com
- Modules: Packages (package.json fields)
  nodejs.org

VIDEOS



NPM Semantic Versioning (SemVer) Explained for Beginners!
CodeLucky

Hand-picked references for this lesson.